

OSGi Configuration Annotations

@ObjectClassDefinition · @AttributeDefinition · @Designate · Factory Configs

1 · @ObjectClassDefinition — the config schema

Lives on a Java @interface file. Declares that this interface is an OSGi Metatype descriptor — it defines the shape of a configuration object. The OSGi framework reads it at bundle startup via a generated METATYPE-INF/metatype.xml and uses it to render the Felix Web Console UI and validate config values.

name	Human-readable label	Felix UI section heading
description	Explanation of the config	Subtitle under the heading
pid	Auto (fully qualified class name)	Override only if needed
factoryPid	(empty = single-instance)	Set via @Designate(factory=true)

Example

```
@ObjectClassDefinition(  
    name      = "My Service Configuration",  
    description = "Settings for MyService"  
)  
public @interface MyConfig {  
    // ... field definitions  
}
```

2 · @AttributeDefinition — individual config fields

Each method on the @interface maps to one config property. The method name becomes the property key (with name substitution). The return type determines the UI widget type. The default clause provides the fallback value.

Return type → UI widget mapping

Return type	UI widget	Key example	Notes
String	Text input	base.url	Single-line text
int / long	Numeric input	timeout	Parsed from string

boolean	Checkbox	enabled	true / false
String[]	Multi-value list	allowed.paths	+ / – row buttons
Enum type	Dropdown select	mode	Use options= to fill list
char[]	Password field	secret.key	Masked in Felix UI

Name substitution rule

OSGi converts method names to property keys: `_` (single underscore) $\rightarrow$ `.` (dot) `__` (double underscore) $\rightarrow$ `_` (literal underscore) `$` $\rightarrow$ removed. So `base_url()` stores as key `base.url`, and `my__key()` stores as `my_key`. This lets any property key be expressed as a valid Java identifier.

Full example

```
@AttributeDefinition(name = "Base URL", description = "API base URL")
String base_url() default "https://api.example.com";
@AttributeDefinition(name = "Timeout (ms)")
int timeout() default 5000;
@AttributeDefinition(name = "Enabled")
boolean enabled() default true;
@AttributeDefinition(name = "Allowed Paths")
String[] allowed_paths() default {"/content", "/apps"};
// Dropdown example
@AttributeDefinition(name = "Mode",
    options = {
        @Option(label="Read-only", value="ro"),
        @Option(label="Read-write", value="rw")
    })
String mode() default "ro";
```

3 • @Designate — link schema to component

`@Designate` sits on the component *class*, not on the *@interface*. It tells the OSGi framework: when resolving config for this component, use the metatype schema defined by `ocd = MyConfig.class`. Without it, Felix shows a raw key/value editor with no labels, types, or defaults.

Without `@Designate`

With `@Designate`

Raw key/value editor. No labels, no types, no defaults, no validation. Painful for ops teams.	Proper form with labelled fields, correct input types, default values pre-filled, tooltips from description=. Schema travels inside the bundle jar.
---	---

What the framework does at runtime

1. Bundle activates → OSGi reads the metatype XML generated into the jar at build time
2. Felix Config Admin creates a config node for the PID (= fully qualified class name by default)
3. When the component activates, Config Admin injects the config object
4. @Designate tells Felix which OCD (ObjectClassDefinition) applies for validation and coercion

Single-instance example

```

@Component(service = MyService.class)
@Designate(ocd = MyConfig.class)
public class MyServiceImpl implements MyService {
    @Activate
    protected void activate(MyConfig cfg) {
        String url = cfg.base_url(); // resolved value or default
        int ms      = cfg.timeout();
        boolean on  = cfg.enabled();
    }
}

```

4 · factory = true — multiple named instances

A factory config lets you run *N separate instances* of the same component simultaneously, each with different config values. Classic use-cases: multiple datasource connections, multiple cache region configs, multiple replication endpoints.

Property	Single-instance	Factory (factory=true)
Config node name	com.example.MyServiceImpl	com.example.MultiServiceImpl~<name>
Instances	Exactly 1	One per config node created
AEM/CRX .config	...MyServiceImpl.config	...~instance1.config, ~instance2.config ...
Felix UI	One entry, no + button	List with + button to add instances

The ~ separator is the OSGi factory config naming convention. Everything before ~ is the factory PID (= the component class name). Everything after ~ is the instance name — any string, used only to differentiate config nodes. OSGi spawns one component instance per matching config node.

Factory config example

```
@Component(service = MultiService.class)
@Designate(ocd = MultiConfig.class, factory = true)
public class MultiServiceImpl implements MultiService {
    @Activate
    protected void activate(MultiConfig cfg) {
        // This runs once per config node found with the factory PID
        // e.g. com.example.MultiServiceImpl~instance1
        //      com.example.MultiServiceImpl~instance2
    }
}
```

5 · @Activate consumption patterns

Pattern	Signature	When to use
1 — Recommended	<code>activate(MyConfig cfg)</code>	Most readable. OSGi proxy injects typed values. Arrays work directly.
2 — Raw Map	<code>activate(Map<String, Object> props)</code>	Dynamic key access. Must cast manually. Keys are raw OSGi keys.
3 — ComponentContext	<code>activate(ComponentContext ctx)</code>	When you also need bundle context or service references in activate.

@Modified works identically — same three signatures. Called when config is updated while the component is already active, without a full deactivate/activate cycle. If you do not declare @Modified, any config update triggers a full component restart.

6 · Felix Web Console mapping (/system/console/configMgr)

@OCD name=	Section heading	Config entry title
@OCD description=	Subtitle under heading	
@AD name=	Field label	Left of each input
@AD description=	Tooltip (?) icon	Hover for detail
default on method	Pre-filled value	When no config node exists

String[] return type	Multi-value list	+ / – row buttons
options= on @AD	Dropdown select	Each @Option fills the list
factory=true	"+" button in list view	Creates new named instance

7 · Build pipeline — how annotations become XML

The annotations do not exist at runtime. bnd (called by maven-bundle-plugin or bnd-maven-plugin) processes them at compile time and writes OSGI-INF/metatype/<pid>.xml into the jar. Felix reads that XML. If bnd processing is missing or the annotation processor is not on the compile classpath, no XML is generated and you silently lose all UI rendering and type coercion.

Required Maven dependencies

```
org.osgi
org.osgi.service.metatype.annotations
provided
org.osgi
org.osgi.service.component.annotations
provided
```